

Chess Visual Relation Classifiers Specification

1. Overview

This document specifies neural network classifiers for visually learning simple chess relations from camera photos of screen-rendered chessboards. The system operates purely visually and does not depend on symbolic chess representations during inference.

Target visual relations (limited set):

- Containment: whether a piece is inside a given square (visual square occupancy).
- Neighbourhood: whether two pieces are adjacent (8-neighbour or 4-neighbour).
- Alignment (optional): whether two pieces lie on same file, rank, or diagonal.

2. System Architecture

Step 1: Board corner detection (keypoint model).
Step 2: Homography warp to canonical square board image.
Step 3: Split into 8x8 square crops.
Step 4: Generate relation training samples (single-square or square-pair).
Step 5: Train independent relation classifiers.

All relation models operate only on local crops or crop pairs.

3. Datasets and References

Board Corner / Geometry Datasets:

- surawut/chessboard-dataset-yolo (HuggingFace):
<https://huggingface.co/datasets/surawut/chessboard-dataset-yolo>
- Roboflow Chessboard Corners dataset:
<https://universe.roboflow.com/chessboarddetect/chessboard-corners-wwivs>

Board / Piece Appearance Datasets:

- MohammedHemed/Chessboard-digital-images_with_fen (HuggingFace):
https://huggingface.co/datasets/MohammedHemed/Chessboard-digital-images_with_fen
- Synthetic Chess Board Images (Kaggle):
<https://www.kaggle.com/datasets/thefamousrat/synthetic-chess-board-images>
- ChessRender360 (Kaggle):
<https://www.kaggle.com/datasets/mmkoya/chessrender360>

4. Model Specifications

ContainmentNet:

- Input: single square crop (e.g., 64x64 or 96x96)
- Backbone: ResNet-18 or small CNN
- Output: binary (contains piece / empty) or multi-class
- Loss: Cross-entropy

NeighbourNet:

- Input: pair of square crops
- Architecture: Siamese CNN (shared backbone) + MLP relation head
- Output: binary (adjacent / not adjacent)
- Loss: Binary cross-entropy

AlignmentNet (optional):

- Input: pair of square crops
- Architecture: Siamese CNN + MLP
- Output: 4-class (none, file, rank, diagonal)
- Loss: Cross-entropy

5. Data Augmentation Strategy

To generalize from clean digital datasets to camera photos of screens, apply augmentations during training:

- Perspective distortion
- Motion blur
- JPEG compression artifacts
- Brightness and contrast shifts
- Glare / bloom simulation
- Moiré / scanline simulation
- Partial occlusions (UI overlays, cursor highlights)

6. Suggested Repository Structure

```
chess_relations/  
  data/  
  models/  
  scripts/  
    01_download_datasets.py  
    02_train_corners.py  
    03_warp_and_crop.py  
    04_build_relation_dataset.py  
    05_train_containment.py  
    06_train_neighbourhood.py  
    07_train_alignment.py  
    08_infer_relations.py
```